Customer Shopping Behavior Analysis

1. Project Overview

This project provides an analysis of customer shopping behavior using transactional data from 3,900 purchases across various product categories. The goal is to uncover insights into spending patterns, customer segments, product preferences, and subscription behavior to guide strategic business decisions.

2. Dataset Summary

- Rows: 3,900

- Columns: 18

- Key Features:

- Customer demographics (Age, Gender, Location, Subscription Status)

- Purchase details (Item Purchased, Category, Purchase Amount, Season, Size, Color)

- Shopping behavior (Discount Applied, Promo Code Used, Previous Purchases, Frequency of Purchases, Review Rating, Shipping Type)

- Missing Data: 37 values in Review Rating column

3. Exploratory Data Analysis using Python

We began with data preparation and cleaning in Python:

● **Data Loading:** Imported the dataset using pandas.

● **Initial Exploration:** Used df.info() to check structure and .describe() for summary statistics.

```
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Customer ID             3900 non-null   int64
 1   Age                     3900 non-null   int64
 2   Gender                  3900 non-null   object
 3   Item Purchased          3900 non-null   object
 4   Category                3900 non-null   object
 5   Purchase Amount (USD)   3900 non-null   int64
 6   Location                3900 non-null   object
 7   Size                    3900 non-null   object
 8   Color                   3900 non-null   object
 9   Season                  3900 non-null   object
 10  Review Rating           3863 non-null   float64
 11  Subscription Status     3900 non-null   object
 12  Shipping Type           3900 non-null   object
 13  Discount Applied        3900 non-null   object
 14  Promo Code Used         3900 non-null   object
 15  Previous Purchases      3900 non-null   int64
 16  Payment Method          3900 non-null   object
 17  Frequency of Purchases  3900 non-null   object
dtypes: float64(1), int64(4), object(13)
```

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied
count	3900.000000	3900.000000	3900	3900	3900	3900.000000	3900	3900	3900	3900	3863.000000	3900	3900	3900
unique	NaN	NaN	2	25	4	NaN	50	4	25	4	NaN	2	6	2
top	NaN	NaN	Male	Blouse	Clothing	NaN	Montana	M	Olive	Spring	NaN	No	Free Shipping	No
freq	NaN	NaN	2652	171	1737	NaN	96	1755	177	999	NaN	2847	675	2223
mean	1950.500000	44.068462	NaN	NaN	NaN	59.764359	NaN	NaN	NaN	NaN	3.750065	NaN	NaN	NaN
std	1125.977353	15.207589	NaN	NaN	NaN	23.685392	NaN	NaN	NaN	NaN	0.716983	NaN	NaN	NaN
min	1.000000	18.000000	NaN	NaN	NaN	20.000000	NaN	NaN	NaN	NaN	2.500000	NaN	NaN	NaN
25%	975.750000	31.000000	NaN	NaN	NaN	39.000000	NaN	NaN	NaN	NaN	3.100000	NaN	NaN	NaN
50%	1950.500000	44.000000	NaN	NaN	NaN	60.000000	NaN	NaN	NaN	NaN	3.800000	NaN	NaN	NaN
75%	2925.250000	57.000000	NaN	NaN	NaN	81.000000	NaN	NaN	NaN	NaN	4.400000	NaN	NaN	NaN
max	3900.000000	70.000000	NaN	NaN	NaN	100.000000	NaN	NaN	NaN	NaN	5.000000	NaN	NaN	NaN

Promo Code Used	Previous Purchases	Payment Method	Frequency of Purchases
3900	3900.000000	3900	3900
2	NaN	6	7
No	NaN	PayPal	Every 3 Months
2223	NaN	677	584
NaN	25.351538	NaN	NaN
NaN	14.447125	NaN	NaN
NaN	1.000000	NaN	NaN
NaN	13.000000	NaN	NaN
NaN	25.000000	NaN	NaN
NaN	38.000000	NaN	NaN
NaN	50.000000	NaN	NaN

Missing Data Handling: Checked for null values and imputed missing values in the Review Rating column using the median rating of each product category.

	0
Customer ID	0
Age	0
Gender	0
Item Purchased	0
Category	0
Purchase Amount (USD)	0
Location	0
Size	0
Color	0
Season	0
Review Rating	37
Subscription Status	0
Shipping Type	0
Discount Applied	0
Promo Code Used	0
Previous Purchases	0
Payment Method	0
Frequency of Purchases	0

dtype: int64

● **Column Standardization:** Renamed columns to **snake case** for better readability and documentation.

```
#column names uniformity
df.columns = df.columns.str.lower()
df.columns = df.columns.str.replace(' ', '_')
df= df.rename(columns={'purchase_amount_(usd)': 'purchase_amount'})
```

```
df.columns
```

```
Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
       'purchase_amount_(usd)', 'location', 'size', 'color', 'season',
       'review_rating', 'subscription_status', 'shipping_type',
       'discount_applied', 'promo_code_used', 'previous_purchases',
       'payment_method', 'frequency_of_purchases'],
      dtype='object')
```

● Feature Engineering: ○ Created **age_group** column by binning customer ages.

```
#feature engineering
# Creating an age_group column

labels = ['Young Adult', 'Adult', 'Middle-aged', 'Senior']
df['age_group'] = pd.qcut(df['age'], q=4, labels = labels)
```

```
df[['age', 'age_group']].head(10)
```

	age	age_group
0	55	Middle-aged
1	19	Young Adult
2	50	Middle-aged
3	21	Young Adult
4	45	Middle-aged
5	46	Middle-aged
6	63	Senior

○ Created **purchase_frequency_days** column from purchase data.

● **Data Consistency Check:** Verified if discount_applied and promo_code_used were redundant; dropped promo_code_used.

● **Database Integration:** Connected Python script to SQLite and loaded the cleaned DataFrame into the database for SQL analysis.

```
pip install SQLAlchemy
```

```
Requirement already satisfied: SQLAlchemy in /usr/local/lib/python3.12/dist-packages (2.0.48)
Requirement already satisfied: greenlet>=1 in /usr/local/lib/python3.12/dist-packages (from SQLAlchemy) (3.3.2)
Requirement already satisfied: typing-extensions>=4.6.0 in /usr/local/lib/python3.12/dist-packages (from SQLAlchemy) (4.15.0)
```

```
from sqlalchemy import create_engine

# SQLite database file will be created in the Colab environment
database_file = 'customer_behavior.db'

# Create a SQLAlchemy engine for SQLite
# 'sqlite:///' specifies a local file-based database
engine = create_engine(f'sqlite:///{database_file}')

# Load the DataFrame into the SQLite database
table_name = 'all_customer'
df.to_sql(table_name, engine, if_exists='replace', index=False)

print(f"DataFrame successfully loaded into table '{table_name}' in the SQLite database '{database_file}'.")
```

4. Data Analysis using SQL (Business Transactions)

We performed structured analysis in SQLite to answer key business questions:

4.1. **Revenue by Gender** – Compared total revenue generated by male vs. female customers.

```
#Total revenue generated by male vs female customers

query = """
SELECT gender, SUM(purchase_amount) AS total_revenue
FROM all_customer
GROUP BY gender
"""
revenue_df = pd.read_sql(query, engine)
print(revenue_df)
```

```
   gender  total_revenue
0  Female          75191
1    Male         157890
```

4.2. **High-Spending Discount Users** – Identified customers who used discounts but still spent above the average purchase amount.

```
#Customers who used a discount but still spent more than the average purchase amount
query = """
SELECT customer_id, purchase_amount
FROM all_customer
WHERE discount_applied = 'Yes' AND purchase_amount >= (SELECT AVG(purchase_amount) FROM all_customer)
"""
cust_df= pd.read_sql(query, engine)
print(cust_df)
```

```
     customer_id  purchase_amount
0              2               64
1              3               73
2              4               90
3              7               85
4              9               97
..           ...              ...
834         1667               64
835         1671               73
836         1673               73
837         1674               62
838         1676               90

[839 rows x 2 columns]
```

4.3. **Top 5 Products by Rating** – Found products with the highest average review ratings.

```
#Top 5 items with highest review rating
query = """
SELECT item_purchased, AVG(review_rating) AS avg_rating
FROM all_customer
GROUP BY item_purchased
ORDER BY avg_rating DESC
LIMIT 5
"""
top_items= pd.read_sql(query, engine)
top_items['avg_rating'] = top_items['avg_rating'].round(2)
print(top_items)
```

```
  item_purchased  avg_rating
0         Gloves        3.86
1        Sandals        3.84
2          Boots        3.82
3            Hat        3.80
4          Skirt        3.78
```

4.4. **Shipping Type Comparison** – Compared average purchase amounts between Standard and Express shipping.

```
#Average purchase amount between standard and express shipping
query = """
SELECT shipping_type, AVG(purchase_amount) AS avg_purchase_amount
FROM all_customer
WHERE shipping_type IN ('Standard', 'Express')
GROUP BY shipping_type
"""
shipping_df = pd.read_sql(query, engine)
shipping_df['avg_purchase_amount'] = shipping_df['avg_purchase_amount'].round(2)
print(shipping_df)
```

```
  shipping_type  avg_purchase_amount
0       Express                60.48
1      Standard                58.46
```

4.5. **Subscribers vs. Non-Subscribers** – Compared average spend and total revenue across subscription status.

```
#Compare average spend and total revenue btwn subscribed vs non
query = """
SELECT subscription_status, COUNT(customer_id) AS total_customers, AVG(purchase_amount) AS avg_spend, SUM(purchase_amount) AS total_revenue
FROM all_customer
GROUP BY subscription_status
ORDER BY total_revenue, avg_spend DESC
"""
spend_df = pd.read_sql(query, engine)
spend_df['avg_spend'] = spend_df['avg_spend'].round(2)
spend_df['total_revenue'] = spend_df['total_revenue'].round(2)
print(spend_df)
```

```
  subscription_status  total_customers  avg_spend  total_revenue
0                 Yes             1053      59.49          62645
1                  No             2847      59.87         170436
```

4.6. **Discount-Dependent Products** – Identified 5 products with the highest percentage of discounted purchases.

```
# Top 5 products with highest percentage of purchase with discount applied
query = """
SELECT item_purchased, (COUNT(CASE WHEN discount_applied = 'Yes' THEN 1 END) * 100.0 / COUNT(*)) AS discount_percentage
FROM all_customer
GROUP BY item_purchased
ORDER BY discount_percentage DESC
LIMIT 5
"""
discount_df = pd.read_sql(query, engine)
discount_df['discount_percentage'] = discount_df['discount_percentage'].round(2)
print(discount_df)
discount_df
```

```
  item_purchased  discount_percentage
0            Hat                50.00
1       Sneakers                49.66
2           Coat                49.07
3        Sweater                48.17
4          Pants                47.37
```

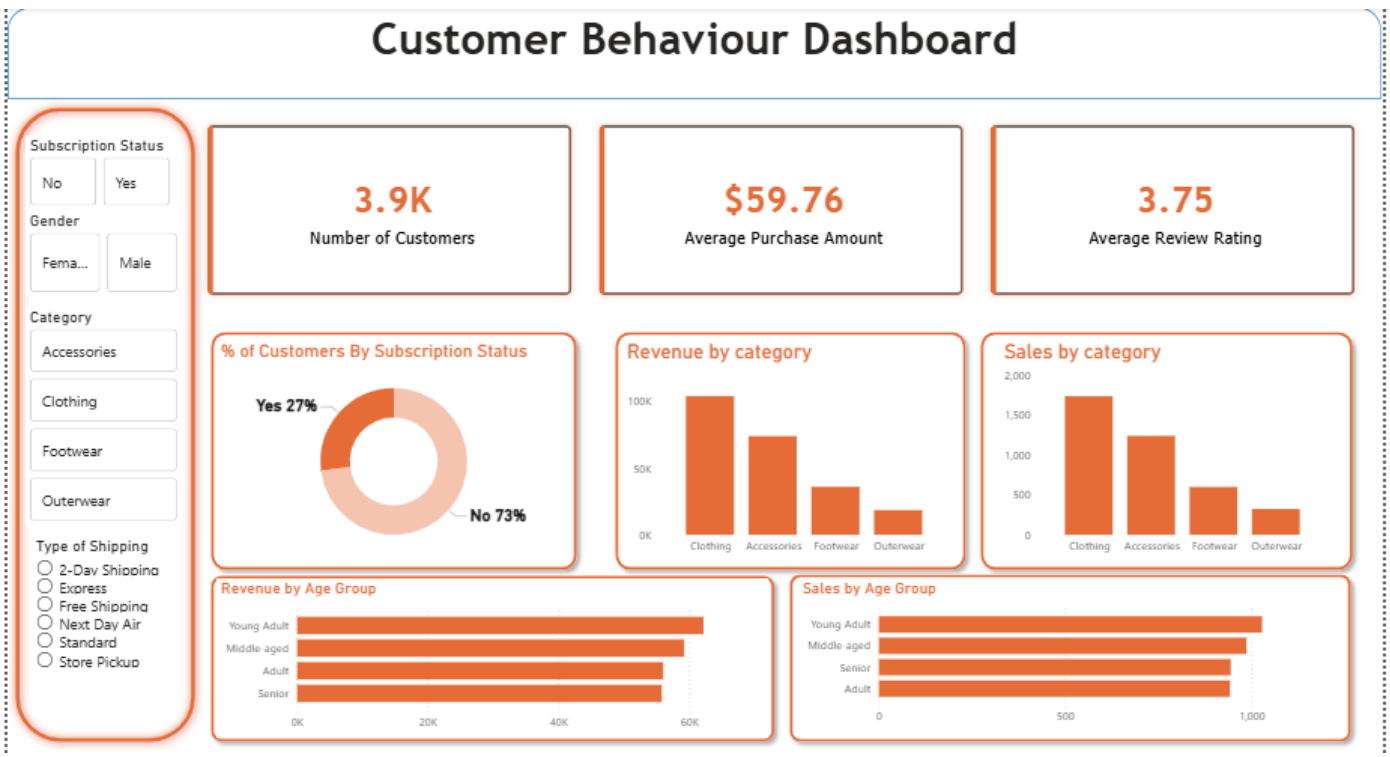
4.7. **Revenue by Age Group** – Calculated total revenue contribution of each age group.

```
#Total revenue by age group
query = """
SELECT age_group, SUM(purchase_amount) AS total_revenue
FROM all_customer
GROUP BY age_group
ORDER BY total_revenue ASC
"""
revenue_df = pd.read_sql(query, engine)
print(revenue_df)
```

```
     age_group  total_revenue
0       Senior          55763
1        Adult          55978
2  Middle-aged          59197
3  Young Adult          62143
```

5. Dashboard in Power BI

Finally, we built an interactive dashboard in **Power BI** to present insights visually.



6. Business Recommendations

1. Convert non-subscribers

Non-subscribers make up 73% of all customers (2,847 customers) and generate $170,436 in revenue compared to $62,645 from subscribers. Despite a nearly identical average spend ($59.87 vs $59.49), the sheer volume gap means a targeted subscription conversion campaign, even converting 10% of non-subscribers could significantly grow recurring revenue.

2. Protect Margins on products with heavy discounts

Hats, Sneakers, Coats, Sweaters, and Pants have discount application rates between 47–50%, meaning nearly half of all purchases in these categories are discounted. The business should evaluate whether these discounts are driving incremental purchases or simply eroding margins on sales that would have happened anyway. A controlled A/B test reducing discounts on one of these products would answer this directly.

3. Double Down on Young Adults

Young Adults generate the most revenue at $62,143, outperforming Seniors ($55,763) and Adults ($55,978) despite likely having lower individual purchasing power. This suggests higher purchase frequency in this segment. Marketing spend and loyalty rewards should be disproportionately directed here to protect and grow this segment.

4. Leverage Express Shipping as a Premium Signal

Express shipping customers have a higher average purchase amount ($60.48) compared to Standard ($58.46). This suggests a correlation between willingness to pay for faster delivery and higher overall spend , these are your premium customers. They should be prioritized for upselling, early access offers, and loyalty programs.

5. Build a Loyalty Programs Targeting High-Value Discount Users

839 customers who used discounts but still spent above the average purchase amount, there is a clear high-value segment that responds to promotions without being purely price-driven. A tiered loyalty program targeting this group could increase retention without requiring continuous discounting.